PROGRAMMING FOR PROBLEM SOLVING (BE01000121)

Prepared By,

Prof. Shraddha Modi

Computer Engineering

LDCE, Ahmedabad

CHAPTER-9
DYNAMIC MEMORY ALLOCATION

Prepared By,
Prof. Shraddha Modi
Computer Engineering
LDCE, Ahmedabad

ROADMAP

- Introduction to Dynamic memory allocation,
- malloc() calloc(), realloc(), free() functions

STATIC VS DYNAMIC

static memory allocation	dynamic memory allocation
memory is allocated at compile time.	memory is allocated at run time.
memory can't be increased while executing program.	memory can be increased while executing program.
used in array.	used in linked list.

DYNAMIC MEMORY ALLOCATION

malloc()	allocates single block of requested memory.
calloc()	allocates multiple block of requested memory.
realloc()	reallocates the memory occupied by malloc() or calloc() functions.
free()	frees the dynamically allocated memory.

INTRODUCTION

- An array is a collection of a fixed number of values. Once the size of an array is declared, you cannot change it.

- Sometimes the size of the array you declared may be insufficient. To solve this issue, you can allocate memory manually during run-time. This is known as dynamic memory allocation in C programming.

- To allocate memory dynamically, library functions are malloc(), calloc(), realloc() and free() are used. These functions are defined in the <stdlib.h> header file.

MALLOC()

- "malloc" stands for memory allocation.
- The malloc() function reserves a block of memory of the specified number of bytes. And, it returns a pointer of void which can be casted into pointers of any form.

Syntax of malloc()

```
ptr = (castType*) malloc(size);
```

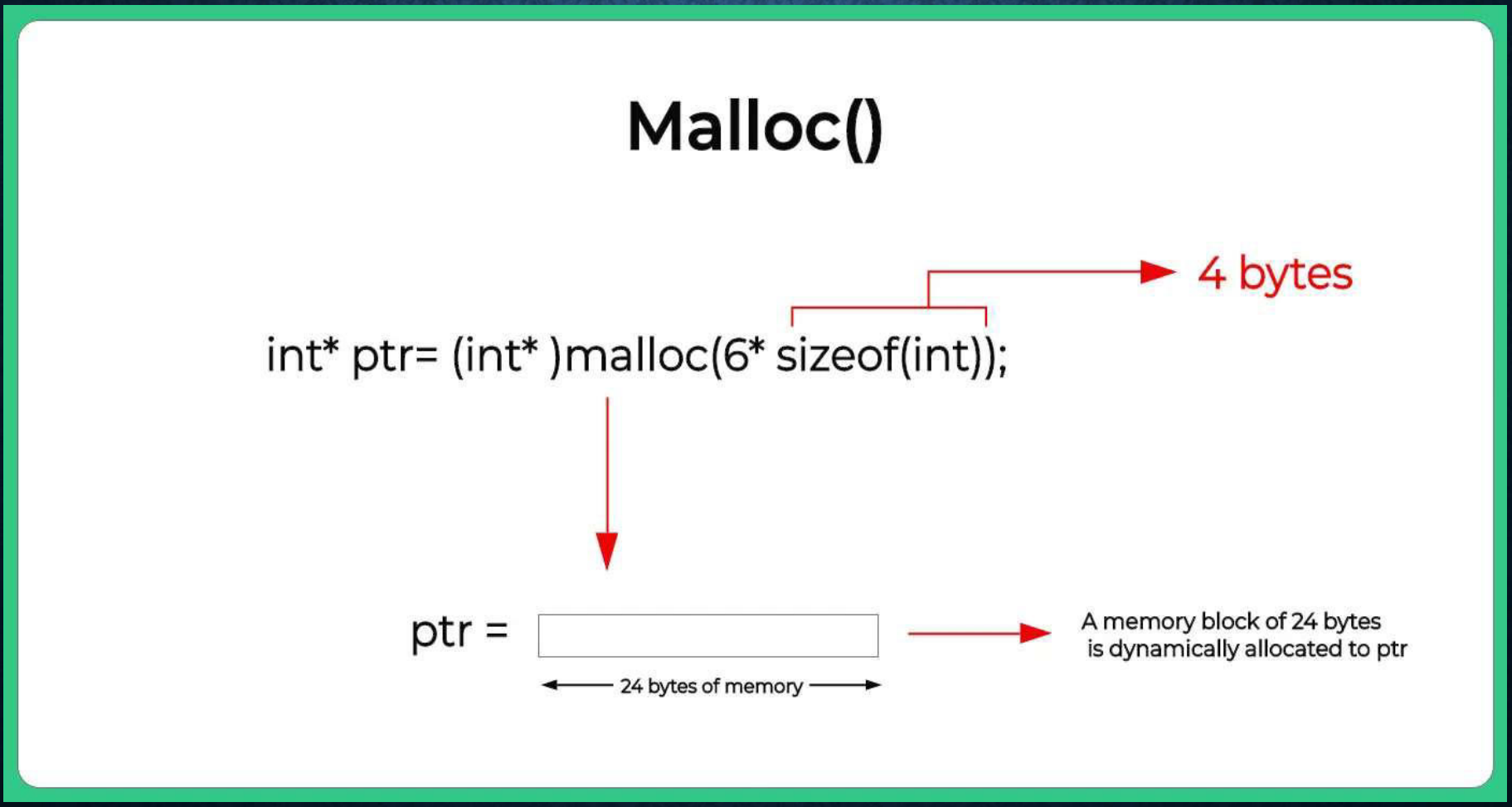
MALLOC()

Example

```
ptr = (float*) malloc(100 * sizeof(float));
```

- The above statement allocates 400 bytes of memory. It's because the size of float is 4 bytes. And, the pointer ptr holds the address of the first byte in the allocated memory.
- The expression results in a NULL pointer if the memory cannot be allocated.

MALLOC()


Malloc()
4 bytes
int* ptr= (int*)malloc(6* sizeof(int));
ptr =
24 bytes of memory
A memory block of 24 bytes
is dynamically allocated to ptr

MALLOC()

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i;
    int *ptr;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Allocate memory for 'n' elements using malloc()
    ptr = (int *)malloc(n * sizeof(int));
```

MALLOC()

```
if (ptr == NULL) {
        printf("Memory
allocation failed.\n");
        exit(1);
    }
    printf("Enter the
elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ptr[i]);
    printf("The    elements    you
entered are: ");
    for (i = 0; i < n; i++) {
        printf("%d ", ptr[i]);
    }
 free(ptr);
    return 0;
}
```

MALLOC()

```
Enter the number of elements: 3
Enter the elements: 1
2
3
The elements you entered are: 1 2 3
```

CALLOC()

- The name "calloc" stands for contiguous allocation.
- The malloc() function allocates memory and leaves the memory uninitialized, whereas the calloc() function allocates memory and initializes all bits to zero.
- calloc() is used to allocate multiple blocks of memory, typically to store an array of elements.

Syntax of calloc()

```
ptr = (castType*)calloc(n, size);
```

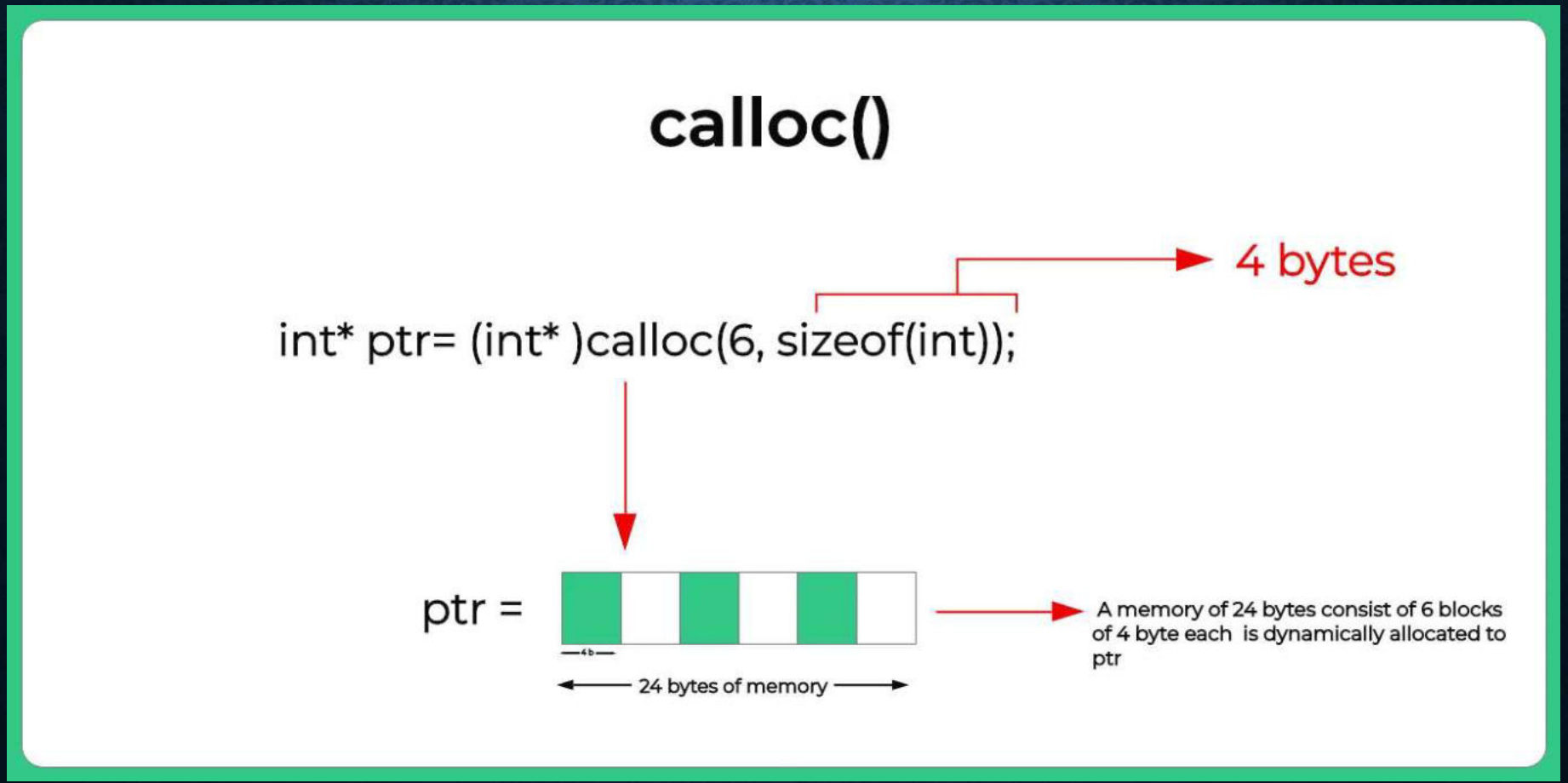
CALLOC()

Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

- The above statement allocates contiguous space in memory for 25 elements of type float.

CALLOC()


calloc()
4 bytes
int* ptr= (int*)calloc(6, sizeof(int));
ptr =
4b
24 bytes of memory
A memory of 24 bytes consist of 6 blocks of 4 byte each is dynamically allocated to ptr

CALLOC()

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i;
    int *ptr;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Allocate memory for 'n' elements using malloc()
    ptr = (int *)calloc(n, sizeof(int));
```

CALLOC()

```
if (ptr == NULL) {
        printf("Memory
allocation failed.\n");
        exit(1);
    }
    printf("Enter the
elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ptr[i]);
    printf("The    elements    you
entered are: ");
    for (i = 0; i < n; i++) {
        printf("%d ", ptr[i]);
    }
 free(ptr);
    return 0;
}
```

CALLOC()

```
Enter the number of elements: 3
Enter the elements: 34
76
89
The elements you entered are: 34 76 89
```

FREE()

- Dynamically allocated memory created with either calloc() or malloc() doesn't get freed on their own.
- You must explicitly use free() to release the space.

Syntax of free()

```
free(ptr);
```

REALLOC()

- If the dynamically allocated memory is insufficient or more than required, you can change the size of previously allocated memory using the realloc() function.

Syntax of realloc()

```
ptr = realloc(ptr, x);
```

REALLOC()

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
int *ptr, i , n1, n2;
printf("Enter size: ");
scanf("%d", &n1);
ptr = (int*) malloc(n1 * sizeof(int));

printf("Addresses of previously allocated memory:\n");
for(i = 0; i < n1; ++i)
printf("%u\n",ptr + i);
```

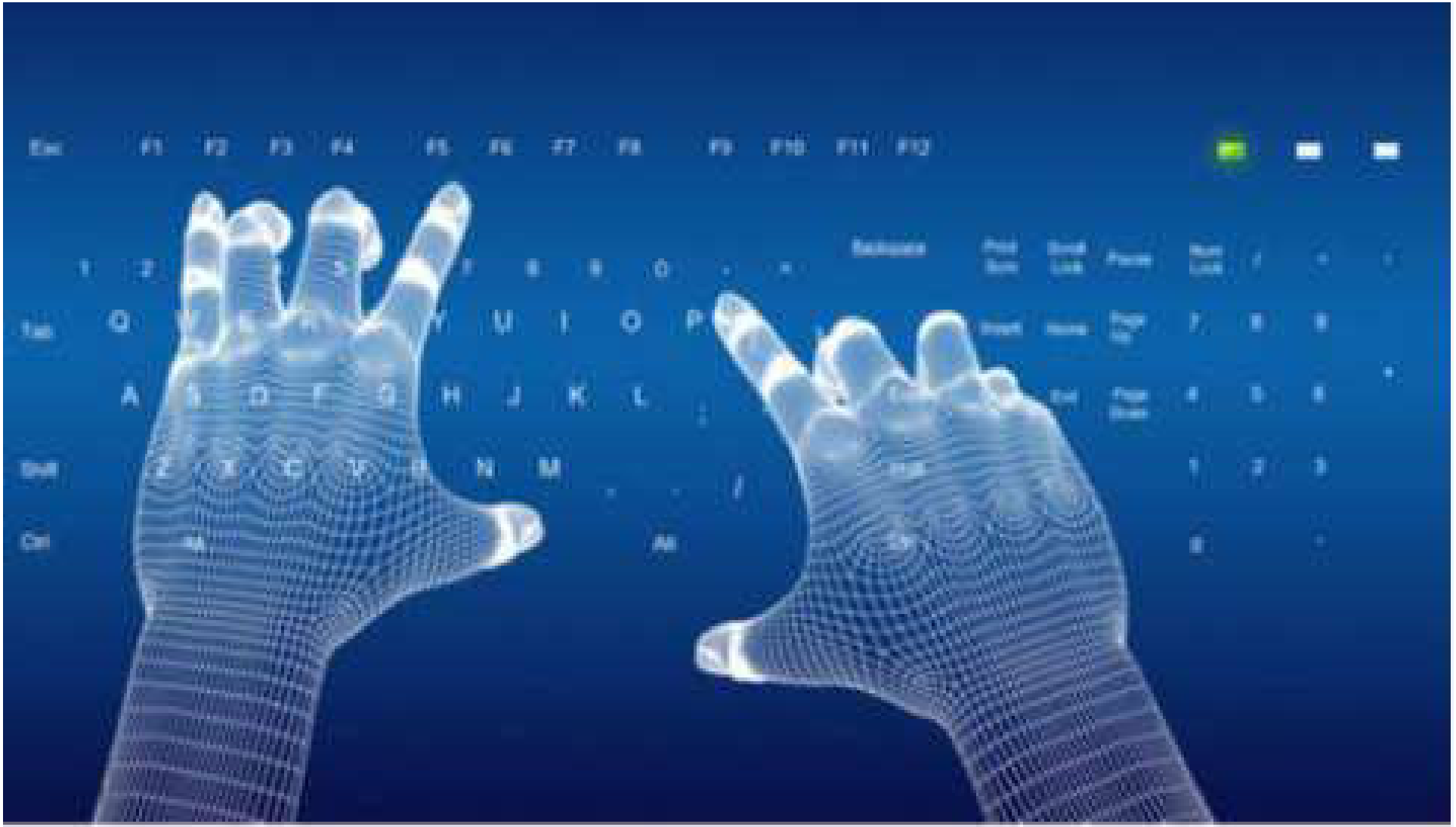
REALLOC()

```
printf("\nEnter the new size:
");
scanf("%d", &n2);
// rellocating the memory
ptr = realloc(ptr, n2 *
sizeof(int));
printf("Addresses of newly
allocated memory:\n");
for(i = 0; i < n2; ++i)
printf("%u\n", ptr + i);
free(ptr);
return 0;
}
```

REALLOC()

```
Enter size: 4
Addresses of previously allocated memory:
12847808
12847812
12847816
12847820

Enter the new size: 6
Addresses of newly allocated memory:
12847808
12847812
12847816
12847820
12847824
12847828
```



Thank You !